1. Why the Saga Pattern Exists

The Core Problem

In microservices:

- Each service owns **its own database**
- Services **cannot share transactions**
- A single business operation spans **multiple services**

Traditional ACID transactions **do not work** across services.

Example Problem

Placing an order requires:

1. Create Order
2. Process Payment
3. Reserve Inventory

If step 2 fails after step 1 succeeds → **inconsistent system state**

2. What Is the Saga Pattern?

The **Saga Pattern** is a **distributed transaction management pattern** used in **microservices architectures**.

It ensures **data consistency** across multiple services **without using a single distributed database transaction**.

Instead of one ACID transaction, a saga is a **sequence of local transactions**, where:

- Each service **updates its own database**, and
- If any step fails, **compensating transactions** are executed to undo previous work.

3. Core Concepts of Saga

1. **Local Transaction**
   A transaction executed within a single service boundary.
2. **Compensating Transaction**
   A rollback operation that semantically undoes a completed local transaction.

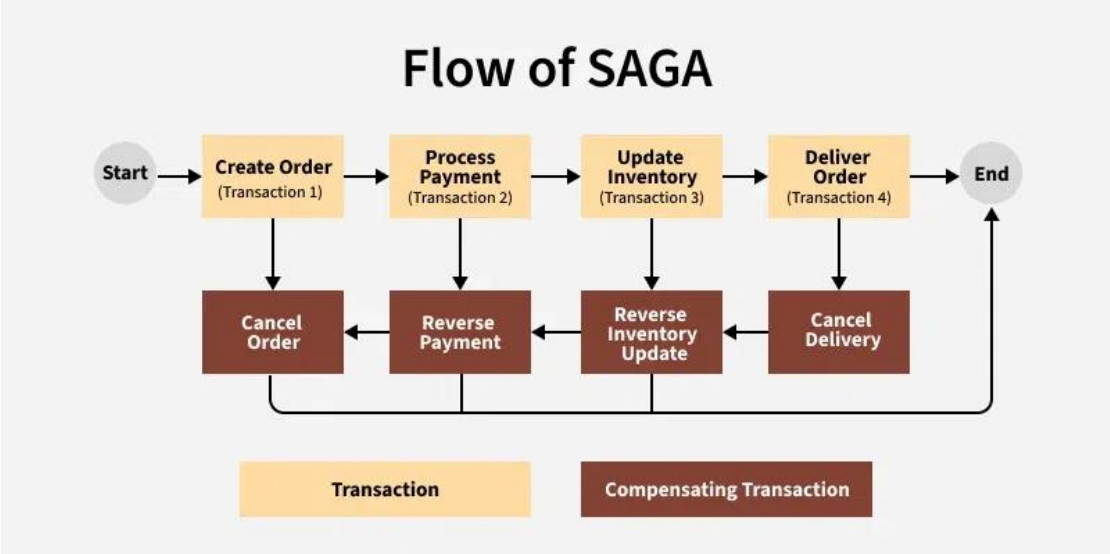
3. **Eventual Consistency**
   Data becomes consistent after all steps (or compensations) complete.

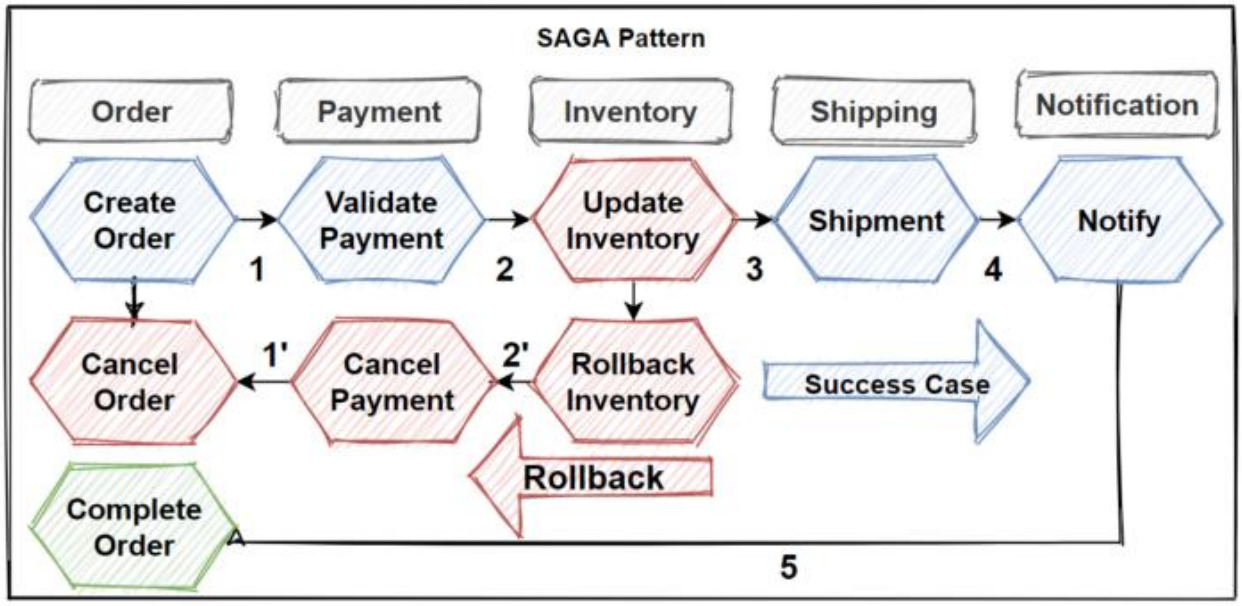
Concept	Meaning
Local Transaction	DB transaction inside one service
Event	Notification that a step completed
Compensation	Undo operation for a completed step
Eventual Consistency	System becomes consistent over time

4. High-Level Saga Flow

Generic Flow

1. Service A completes its local transaction
2. Service A publishes an event
3. Service B reacts and executes its transaction
4. If all steps succeed → Saga completes
5. If any step fails → compensations are triggered





5. Saga Execution Models(Types of Saga Patterns)

There are two implementation models:

1. Choreography-based Saga
2. Orchestration-based Saga

6. Orchestration-based Saga

- A **central orchestrator** controls the workflow
- Sends commands and decides next steps
- Easier to monitor and manage

Example:
Order Service → Payment Service → Inventory Service

How It Works (Step-by-Step)

1 Client requests order creation

2 Saga Orchestrator starts the saga

3 Orchestrator calls Order Service

4 Order Service responds success

5 Orchestrator calls Payment Service

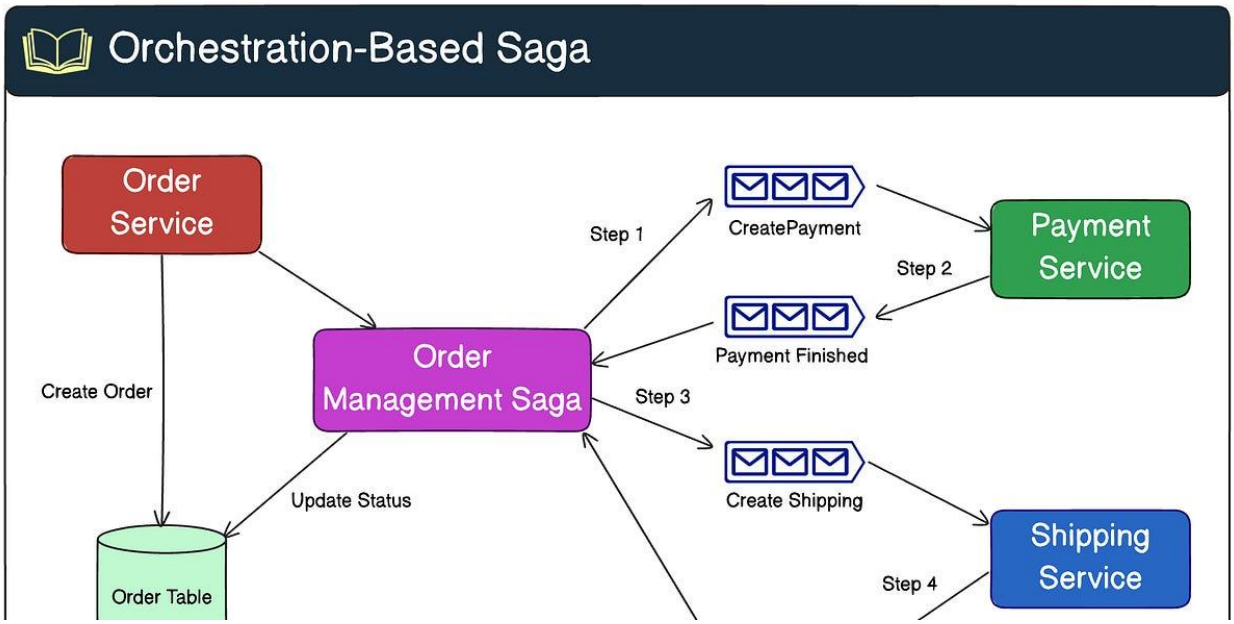
6 Payment fails

7 Orchestrator triggers compensation:

- Cancel Order

When to Use

- Complex workflows
- Clear control flow needed
- Easier monitoring & debugging



7. Choreography-based Saga (Event-Driven)

- No central coordinator
- Services communicate via **events**
- Each service reacts to events and emits new ones

Example:
OrderCreated → PaymentCompleted → InventoryReserved

How It Works (Step-by-Step)

1 Order Service creates an order

2 Order Service publishes OrderCreated

3 Payment Service listens → processes payment

4 Payment Service publishes PaymentSucceeded

5 Inventory Service listens → reserves stock

6 Inventory Service publishes InventoryReserved

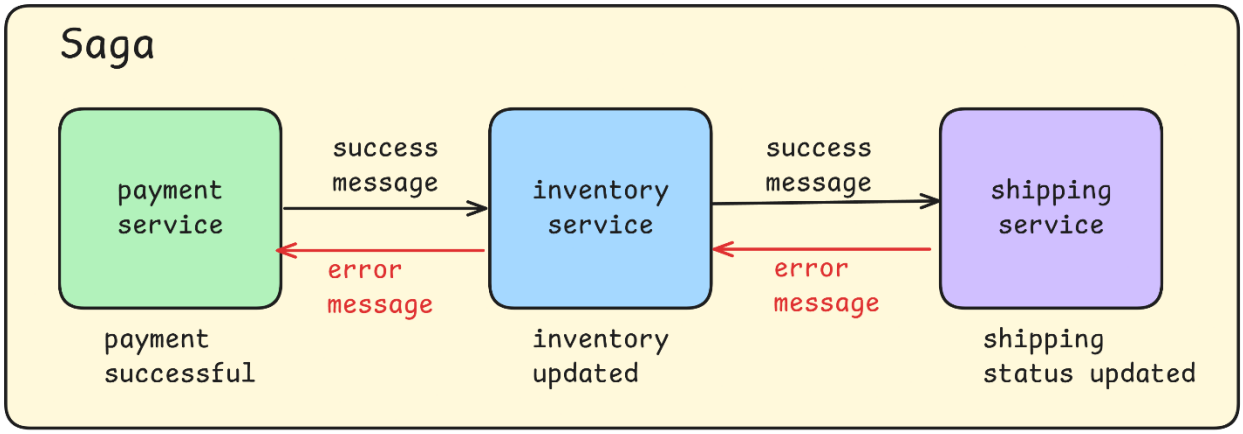
7 Saga completes

Failure Scenario

- Payment fails → publishes PaymentFailed
- Order Service listens → cancels order

When to Use

- Small to medium workflows
- Highly decoupled services
- Event-driven systems



ScalableThread.com

8. Compensation Transactions (Critical Concept)

What Is Compensation?

A compensation transaction undoes a completed step.

Forward Action	Compensation
Create Order	Cancel Order

Forward Action	Compensation
Charge Payment	Refund Payment
Reserve Stock	Release Stock

Key Rules

- **Compensations must be idempotent**
- **Compensations may not fully restore state (business decision)**
- **They are executed only on failure**

Simple Example (E-commerce)

1. **Create Order**
2. **Reserve Payment**
3. **Reserve Inventory**

Failure case:
If Inventory reservation fails →
→ Refund Payment
→ Cancel Order

Advantages

- Scales well in distributed systems
- No tight coupling between services
- Works well with event-driven architecture

Challenges

- Complex compensation logic
- Harder debugging and tracing
- Requires strong observability (logs, tracing)

9.Use Saga when:

- You have **multiple microservices**
- Each service has its **own database**
- Strong consistency is not mandatory
- Business rollback is acceptable
- Distributed business workflows
- Multiple services updating data
- Event-driven architectures
- High availability systems

When You Should NOT

- Single service transaction
- Simple CRUD operations
- Strong consistency requirements

10. Key Takeaways

- Saga replaces distributed transactions
- Uses local transactions + events
- Supports eventual consistency
- Compensation is mandatory
- Two models: Choreography & Orchestration
- Essential for real-world microservices